



UNIVERSITÄT POTSDAM
MATHEMATISCH-NATURWISSENSCHAFTLICHE
FAKULTÄT
INFORMATIK & COMPUTATIONAL SCIENCE

Konzeption

Konzeption und Implementierung eines AST-basierten Frameworks zur
Identifizierung und Analyse von Test-Artefakten in C++ und CMake
Forschungs-Repositories

Gutachter:innen:

—

Autor:

Oskar Soth (806965)

4. Mai 2026

1 Einleitung

Die Suche nach Test-Artefakten in komplexen C++- und CMake-Strukturen lässt sich mit simplen textbasierten Suchmethoden (wie Regulären Ausdrücken) nicht zuverlässig lösen. Verschachtelte Makros, auskommentierter Code oder mehrzeilige Befehle führen bei rein lexikalischen Ansätzen unweigerlich zu hohen Raten an False Positives. Aus diesem Grund wird in dieser Arbeit ein strukturierter Ansatz gewählt. Anstatt den Code als flachen Text zu betrachten, wird er mithilfe eines Parsers in seine zugrundeliegende syntaktische Struktur (Abstract Syntax Tree) überführt. Ziel dieser Arbeit ist es, auf Basis dieses Strukturbaums ein Framework zu konzipieren, das in der Lage ist, die verteilten Artefakte semantisch zu verknüpfen. Dabei soll im Rahmen der Evaluation untersucht werden, inwiefern dieser AST-basierte Ansatz eine zuverlässigere und präzisere Identifikation von Test-Artefakten ermöglicht als eine rein textbasierte Methode.

2 Stand der Technik

Um die Konzeption eines neuen, AST-basierten Frameworks fundiert zu begründen, ist es zunächst erforderlich, die etablierten Verfahren zur statischen Code-Analyse kritisch zu betrachten. Dieses Kapitel evaluiert den aktuellen Stand der Forschung und beleuchtet die technologischen Grenzen existierender Ansätze bei der Identifikation von Test-Artefakten. Dafür wird zunächst die theoretische Notwendigkeit einer strukturellen anstelle einer rein lexikalischen Analyse hergeleitet. Daraufhin werden die Schwächen etablierter Allzweck-Analysertools im spezifischen Kontext von Test-Code aufgezeigt, bevor moderne Parser-Technologien für das Vorhaben verglichen werden.

2.1 Grenzen lexikalischer Analyse

Lexikalische Suchverfahren sind mathematisch auf reguläre Sprachen beschränkt und können die tief verschachtelten Strukturen von C++ und CMake nicht zuverlässig erfassen. Wie *Aho et al.* [1, S. 209 ff.] darlegen, erfordern bereits grundlegende Eigenschaften C-ähnlicher Sprachen die Einbeziehung semantischen Kontexts, um Bezeichner (Namen von Variablen oder Typen) korrekt aufzulösen. C++ ist daher im Kern als kontextsensitiv (Typ-1) einzustufen, was einen Parser-basierten Ansatz, der den Code in einen Abstract Syntax Tree (AST) überführt, zwingend erforderlich macht.

2.2 Schwächen aktueller Analyse-Tools bei Test-Code

Die dargelegten theoretischen Beschränkungen lexikalischer Analysen manifestieren sich in der Praxis als Nachteile bei der Zuverlässigkeit der Artefakterkennung. *Malm et al.* [2] untersuchten den Einsatz moderner statischer Analysewerkzeuge wie Clang-Tidy, Clang Static Analyzer und Cppcheck auf Test-Code und stellten fest, dass diese Werkzeuge ohne spezifische Anpassungen oft unpräzise Ergebnisse liefern. Obwohl sich die Untersuchung primär auf die Detektion von Softwarefehlern konzentriert, lassen sich die identifizierten Ursachen direkt auf die Problematik der automatisierten Erkennung von Test-Artefakten übertragen. Ein wesentliches Hindernis ist der Mangel an kontextuellen Informationen. Da Test-Code häufig framework-spezifische Designentscheidungen und Makro-Expansionen nutzt, werden diese von allgemeinen Analyse-Tools oft als Rauschen fehlinterpretiert oder gar nicht erst als Test-Logik erkannt. *Malm et al.* [2] heben hervor, dass eine präzise Analyse ein tiefes Verständnis der zugrundeliegenden

Framework-Semantik erfordert – etwa die Erkennung von benutzerdefinierten Assertionen, die für den Kontrollfluss entscheidend sind. Darüber hinaus wird die konzeptionelle Überlegenheit strukturorientierter Verfahren gegenüber rein muster- oder zeilenbasierten Ansätzen deutlich. Während einfache, auf regulären Ausdrücken basierende Prüfungen schnell sind, beschränken sie sich auf syntaktische Oberflächenmerkmale. *Malm et al.* [2] argumentieren hingegen, dass für eine verlässliche Analyse Abstraktionsformate wie der Abstract Syntax Tree notwendig sind, um konsistente Ergebnisse zu erzielen. Für die vorliegende Arbeit bedeutet dies: Eine zuverlässige Erkennung von Test-Artefakten kann nicht allein durch das Scannen nach Textmustern erfolgen. Stattdessen ist ein Tool erforderlich, das die hierarchische Struktur des Build-Systems und des Quellcodes versteht, um framework-spezifische Artefakte sicher von gewöhnlichem Code zu differenzieren.

2.3 Vergleich von Parser-Technologien

Um die Eignung verschiedener Parser-Technologien für die sprachübergreifende Analyse zu bewerten, müssen diese hinsichtlich ihrer Architektur, ihrer Sprachunterstützung und ihrer Robustheit verglichen werden. Insbesondere bei der Analyse von heterogenen Repositories, die unvollständigen Code oder noch nicht konfigurierte Build-Skripte enthalten können, ist die Fehlertoleranz des Parsers ein entscheidendes Kriterium [3]. Tabelle 1 stellt die relevantesten Technologien im Kontext dieser Arbeit gegenüber.

Technologie	Analyse-Ansatz	Fokus-Sprachen	Ausgabeformat	Fehlertol.
ANTLR (v4) [3]	ALL(*)	C++, Java, Python	AST	Nein
JavaCC [3]	LL(k)	Primär Java, C++	AST / Code	Nein
srcML [4]	XML-Parsing	C/C++, Java, C#	XML (DOM)	Bedingt
Tree-sitter [3, 5]	GLR	C++, CMake	CST / JSON	Ja

Tabelle 1: Gegenüberstellung etablierter Parser-Technologien zur statischen Code-Analyse.

Die in Tabelle 1 aufgeführten Werkzeuge basieren auf unterschiedlichen theoretischen Parsing-Strategien. Während Top-Down-Verfahren wie LL(*) (ANTLR) versuchen, die Grammatikregeln ausgehend vom Startsymbol abzuleiten, verfolgt Tree-sitter einen Bottom-Up-Ansatz (LR) [1, S. 212, 233-234]. Letzterer gilt mathematisch als mächtiger für die Handhabung komplexer Sprachkonstrukte und Links-Rekursionen [1, S. 234], wie sie in C++ häufig vorkommen.

Wie aus der Gegenüberstellung hervorgeht, scheiden klassische Compiler-Parser wie ANTLR für diesen spezifischen Anwendungsfall aus, da sie bei syntaktischen Fehlern im Quellcode den Analyseprozess in der Regel abbrechen [3].

Ein entscheidendes Alleinstellungsmerkmal von Tree-sitter ist die Nutzung des Generalized LR (GLR) Algorithmus. Wie *Clem et al.* [6] darlegen, ermöglicht dieser es, nicht-deterministische Grammatiken effizient zu verarbeiten. Im Gegensatz zu herkömmlichen LR(1)-Parsern, die auf einen Lookahead von lediglich einem Token beschränkt sind, kann Tree-sitter durch GLR syntaktische Mehrdeutigkeiten auflösen, indem mehrere Parse-Pfade simultan verfolgt werden [5].

In Kombination mit der Fähigkeit zum inkrementellen Parsen ermöglicht dies eine performante und fehlertolerante Analyse. Diese Robustheit ist zwingend erforderlich, um auch in unvollstän-

digen oder fehlerhaften Test-Strukturen verlässliche Artefakt-Beziehungen aufzubauen, ohne dass ein erfolgreicher Kompiliervorgang als Voraussetzung dienen muss.

3 Anforderungsanalyse und Methodik

In diesem Kapitel wird das systematische Vorgehen bei der Entwicklung des Analysetools beschrieben. Im ersten Teil werden die Anforderungen an das Artefakt dargelegt. Im zweiten Teil wird auf die gewählte Forschungsmethodik eingegangen. Da das Ziel dieser Arbeit die Schaffung eines neuen Artefakts zur Lösung eines praktischen Problems ist, wird ein Ansatz aus dem Design Science Research (DSR) gewählt.

3.1 Anforderungsanalyse

Zur strukturierten Erfassung der Anforderungen an das Artefakt wird ein methodischer Ansatz verfolgt. Die Anforderungen werden in funktionale und nicht-funktionale Anforderungen unterteilt und mittels der MoSCoW-Methode priorisiert.

3.1.1 Funktionale Anforderungen

Die funktionalen Anforderungen definieren den Kern des AST-basierten Frameworks. Tabelle 2 gibt eine Übersicht der priorisierten Features.

ID	Feature	Prio	Beschreibung
F01	AST-Parsing	Must	Erzeugung eines validen AST/CST für C++ und CMake mittels <i>Tree-sitter</i> .
F02	Strukturelle Suche	Must	Identifikation von Artefakten über AST-Knotentypen statt Regex.
F03	Cross-Language-Mapping	Must	Mapping von C++-Test-Artefakten mit CMake-Zielen.
F04	Berichtsausgabe	Should	Export der gefundenen Artefakte in geeignetem Format.

Tabelle 2: Funktionale Anforderungen (MoSCoW-Priorisierung)

3.1.2 Nicht-funktionale Anforderungen

Neben den funktionalen Anforderungen muss das Artefakt folgende Qualitätskriterien erfüllen:

Fehlertoleranz: Das System muss in der Lage sein, Analysen auf unvollständigem Code oder in Umgebungen mit fehlenden Abhängigkeiten durchzuführen.

Modularität: Die Architektur muss durch das Einbinden neuer Grammatiken auf weitere Programmiersprachen erweiterbar sein.

Performanz: Die AST-Generierung muss mittels inkrementellem Parsen effizient erfolgen, um auch große Repositories in akzeptabler Zeit zu analysieren.

3.1.3 Abgrenzung

Um den Umfang der Arbeit im vorgegebenen Zeitrahmen zu halten, wird der Fokus explizit eingeschränkt: Das Artefakt dient der **Identifikation und Analyse**. Die Ausführung der

Tests oder eine tiefgehende Prüfung der Build-Umgebung auf Korrektheit (z.B. vollständige Compiler-Simulation) ist nicht Teil dieser Arbeit.

3.2 Wahl der Forschungsmethodik

Die Auswahl der konkreten Methodik erfolgt auf Basis des Vergleichs-Frameworks von *Venable et al. (2017)* [7]. In der Gegenüberstellung verschiedener DSR-Ansätze wird die Systems Development Research Methodology (SDRM) nach *Nunamaker et al. (1990)* [8] als besonders geeignet für Projekte identifiziert, bei denen die Entwicklung eines funktionalen IT-Systems im Vordergrund steht. Gemäß der technologischen Auswahlregeln (Rule O/P-1) ist SDRM die bevorzugte Wahl, wenn das Forschungsergebnis ein IT-System sein soll.

3.3 Der Forschungsprozess nach SDRM

Um die für die Identifikation notwendigen Indikatoren im Quellcode und in den Build-Dateien präzise zu bestimmen, wird ein kombinierter Ansatz verfolgt. Als fachliche Ausgangsbasis dienen dabei die bereits im SWORDS-Projekt [9] etablierten textbasierten Heuristiken. Diese werden im Rahmen dieser Arbeit auf ihre strukturelle Entsprechung innerhalb des Abstract Syntax Trees hin analysiert und in entsprechende Suchmuster überführt. Ergänzend dazu erfolgt eine explorative Analysephase, in der repräsentative Stichproben aus Forschungs-Repositories manuell auf weitere strukturelle Merkmale untersucht werden, die über rein lexikalische Muster hinausgehen. Die so zusammengesetzten Muster bilden die Grundlage für die Definition der AST-Suchheuristiken und die Auswahl der relevanten Knotentypen.

Der daraus resultierende Entwicklungsprozess gliedert sich in Anlehnung an *Nunamaker et al.* [8] in fünf Phasen:

Forschungsschritt	Forschungsaspekte
1. Entwicklung eines konzeptionellen Rahmens	• Eine aussagekräftige Forschungsfrage formulieren. • Systemfunktionalitäten und Anforderungen untersuchen. • Systementwicklungsprozesse/-verfahren verstehen.
2. Systemarchitektur entwickeln	• Ein einzigartiges Architekturdesign für Erweiterbarkeit, Modularität usw. entwickeln. • Funktionalitäten der Systemkomponenten und deren Wechselbeziehungen definieren.
3. Analyse & Systemdesign	• Datenbank-/Wissensbasis-Schema und Prozesse zur Ausführung von Systemfunktionen entwerfen. • Alternative Lösungen entwickeln und eine Lösung auswählen.
4. Bau des (Prototyp-)Systems	• Durch den Systembauprozess Erkenntnisse über Konzepte, Framework und Design gewinnen. • Einblicke in die Probleme und die Komplexität des Systems gewinnen.

Tabelle 3 – Fortsetzung von vorheriger Seite

Forschungsschritt	Forschungsaspekte
5. Beobachtung & Evaluation des Systems	• Nutzung des Systems durch Fallstudien und Feldstudien beobachten. • System durch Laborexperimente oder Feldexperimente evaluieren. • Neue Theorien/Modelle basierend auf Beobachtung und Experimenten der Systemnutzung entwickeln. • Gewonnene Erfahrungen konsolidieren.

Tabelle 3: Methodik der Design Science Research (*Venable et al.* [7] nach *Nunamaker et al.* [8])

Die im zweiten Schritt des SDRM-Prozesses entwickelte Systemarchitektur ist in Abbildung 1 schematisch dargestellt. Sie verdeutlicht den modularen Aufbau des Frameworks sowie den Datenfluss von der Aufnahme des Quellcodes bis hin zur Generierung der Analyseberichte. Zentral ist hierbei die Abstraktion durch die Tree-sitter-Bibliothek, welche die Grundlage für die anschließenden sprachspezifischen Queries bildet.

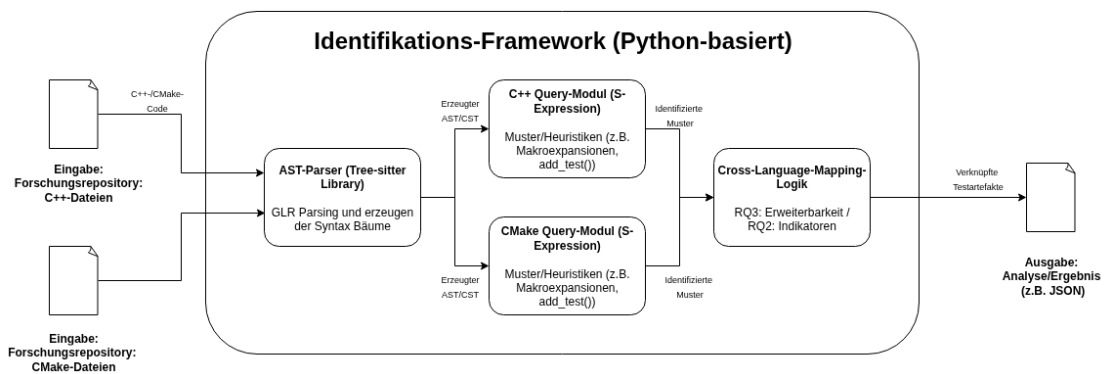


Abbildung 1: Schematische Darstellung der Systemarchitektur und des funktionalen Ablaufs des AST-basierten Identifikations-Frameworks.

4 Evaluationsdesign

Das Ziel dieser Evaluation ist der methodische Nachweis der konzeptionellen Überlegenheit eines AST-basierten Ansatzes gegenüber rein textbasierten Heuristiken bei der Identifikation von Test-Artefakten in Forschungssoftware.

4.1 Evaluationsziele und Forschungsfragen

Die Evaluation zielt auf die empirische Validierung der im Exposé aufgestellten Forschungsfragen ab. Im Fokus stehen dabei insbesondere folgende Aspekte:

- **Effektivität:** Inwiefern verbessert der AST-basierte Ansatz die Erkennungsrate (Recall) und die Genauigkeit (Precision) im Vergleich zum textbasierten Ansatz?
- **Strukturelle Validität:** Können test-relevante Indikatoren in Build-Dateien und Quellcode durch die strukturelle Analyse zuverlässiger extrahiert werden?

- **Erweiterbarkeit:** Es soll evaluiert werden, inwieweit das gewählte Systemdesign und die Abstraktion durch Tree-sitter dazu beitragen, das Framework mit geringem Aufwand auf weitere Programmiersprachen zu übertragen.

4.2 Datengrundlage

Für die Evaluation wird ein vordefinierter Datensatz verwendet, der für diese Arbeit von Dr. Mario Frank zur Verfügung gestellt wurde. Dieser Datensatz umfasst 206 Forschungs-Repositories, die im Journal of Open Source Software (JOSS) publiziert wurden. Das zentrale Auswahlkriterium für diese Stichprobe war das Vorhandensein des „C++“-Tags. Durch die Herkunft aus JOSS ist sichergestellt, dass es sich um reale, wissenschaftlich relevante Softwareprojekte handelt, wodurch der Datensatz eine repräsentative Grundlage für die praxisnahe Überprüfung der AST-basierten Heuristiken bildet.

4.3 Vergleichsmethodik

Um eine objektive Vergleichbarkeit zu gewährleisten, wird ein *Differential Testing* durchgeführt. Anstatt einer manuellen Vollerhebung über den gesamten Datensatz wird das folgende iterative Vorgehen gewählt:

1. **Lexikalische Analyse** (Baseline): Anwendung des im SWORDS-Projekt [9] entwickelten textbasierten Analysewerkzeugs. Dieses Tool nutzt rein lexikalische Heuristiken zur Identifikation von Test-Artefakten und definiert die Baseline für diese Evaluation.
2. **AST-Analyse:** Anwendung des zu implementierenden AST-basierten Frameworks auf denselben Datensatz.
3. **Identifikation der Differenzmenge:** Automatisierter Vergleich der Ergebnismengen zur Identifikation der Differenzmenge.
4. **Manuelle Validierung:** Fokus der manuellen Evaluation auf die Differenzmenge. Hierdurch wird der Aufwand der manuellen Validierung auf diejenigen Fälle konzentriert, in denen beide Analyseansätze unterschiedliche Ergebnisse liefern, was die semantischen Unterschiede der Methoden (lexikalisch vs. strukturell) isoliert.

Die Quantifizierung der Ergebnisse erfolgt anhand der im Buch *Information Retrieval* [10] etablierten Metriken Precision und Recall. Diese ermöglichen eine präzise Messung der Fehlerraten, unterteilt in *False Positives* und *False Negatives*.

4.4 Evaluationsdesign der sprachübergreifenden Analyse

Um die Validität der AST-basierten Identifikation in C++-Quelltexten sicherzustellen, ohne eine manuelle Vollerhebung vornehmen zu müssen, wird ein zweistufiges Vorgehensmodell gewählt:

1. **Automatisierte Cross-Language-Validierung:** Da das Framework sowohl CMake- als auch C++-Artefakte analysiert, dient die CMake-Konfiguration als Ground Truth. Sofern ein Test-Target in den CMake-Dateien explizit definiert ist (*add_test*), muss das Framework im korrespondierenden C++-Quelltext ein entsprechendes Test-Artefakt identifizieren. Dabei wird berücksichtigt, dass CMake-Testdefinitionen (z.B. *add_test()*) technisch auch Nicht-C++-Artefakte (z. B. Python- oder Shell-Skripte) aufrufen können. Um die

Validität des Cross-Language-Mappings nicht zu verfälschen, werden solche Fälle im Rahmen der automatisierten Validierung identifiziert und als 'out of scope' markiert. Dieses Mapping ermöglicht eine automatisierte Prüfung der Sensitivität (Recall) über den gesamten Datensatz hinweg.

2. **Qualitative Fallstudien (Case Studies):** Für komplexe C++-Konstrukte, die über einfache CMake-Referenzen hinausgehen (z. B. template-basierte Test-Cases oder komplexe Makro-Strukturen), erfolgt eine ergänzende, manuelle Analyse. Hierfür werden repräsentative Repositories unterschiedlicher Komplexitätsgrade ausgewählt. Diese Stichproben dienen der qualitativen Verifizierung der AST-basierten Analyseergebnisse und erlauben eine detaillierte Fehleranalyse bei komplexen Sprachkonstrukten.

Dieses hybride Design kombiniert die Skalierbarkeit der automatisierten Validierung mit der notwendigen methodischen Tiefe der manuellen Fallstudien, um die Eignung des AST-Ansatzes auch für komplexe C++-Umgebungen nachzuweisen.

4.5 Evaluierung der Erweiterbarkeit und Modularität

Zur Beantwortung der dritten Forschungsfrage bezüglich der softwarearchitektonischen Erweiterbarkeit wird eine qualitative Bewertung der Systemstruktur vorgenommen. Da das Framework auf einer modularen Architektur basiert und die Parsing-Logik durch Tree-sitter abstrahiert wird, erfolgt die Evaluation in Form einer Machbarkeitsanalyse (Proof of Concept). Hierbei wird exemplarisch dargelegt, welche Schritte notwendig sind, um eine zusätzliche Programmiersprache (z. B. Python) in das bestehende Framework zu integrieren. Bewertet wird dabei insbesondere der Implementierungsaufwand für neue AST-Queries im Vergleich zur bestehenden C++- und CMake-Logik, um die angestrebte Sprachagnostik des Kern-Systems nachzuweisen.

Literatur

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Addison Wesley, August 2006. [Online]. Available: <https://dl.acm.org/doi/book/10.5555/1177220>
- [2] J. Malm, E. Enoiu, M. A. Naser, B. Lisper, Z. Porkoláb, and S. Eldh, “An evaluation of general-purpose static analysis tools on c/c++ test code,” in *2022 48th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, Aug. 2022, p. 133–140. [Online]. Available: <https://ieeexplore.ieee.org/document/10011514/>
- [3] A. Latif, F. Azam, M. W. Anwar, and A. Zafar, “Comparison of leading language parsers – antlr, javacc, sablecc, tree-sitter, yacc, bison,” in *2023 13th International Conference on Software Technology and Engineering (ICSTE)*, 2023, pp. 7–13.
- [4] J. I. Maletic and M. L. Collard, “Exploration, analysis, and manipulation of source code using srcml,” in *Proceedings of the 37th International Conference on Software Engineering - Volume 2*, ser. ICSE ’15. IEEE Press, 2015, p. 951–952.
- [5] M. Brunsfeld *et al.*, “tree-sitter/tree-sitter: v0.26.3,” Dec. 2025. [Online]. Available: <https://doi.org/10.5281/zenodo.17921700>
- [6] T. Clem and P. Thomson, “Static analysis at github,” *Queue*, vol. 19, no. 4, p. 42–67, Aug. 2021.
- [7] J. R. Venable, J. Pries-Heje, and R. L. Baskerville, “Choosing a design science research methodology,” *ACIS 2017 Proceedings*, Jan. 2017. [Online]. Available: <https://aisel.aisnet.org/acis2017/112>
- [8] J. Nunamaker and M. Chen, “Systems development in information systems research,” in *Twenty-Third Annual Hawaii International Conference on System Sciences*, vol. 3, Jan. 1990, p. 631–640 vol.3. [Online]. Available: <https://ieeexplore.ieee.org/document/205401/>
- [9] A. Devkate and A.-L. Lamprecht, “SWORDS template UP,” Aug. 2024. [Online]. Available: <https://github.com/Software-Engineering-Group-UP/SWORDS-template-UP/releases/tag/v1.0.0>
- [10] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008, ch. 8, pp. 154–155. [Online]. Available: <http://nlp.stanford.edu/IR-book/information-retrieval-book.html>